

Умные указатели

Умные указатели - классы, которые оборачивают обычные указатели и автоматически управляют памятью.

- Впервые появились в C++11
- Реализуют концепцию RAI

RAI и концепция умных указателей

RAI - Resource Acquisition Is Initialization

Ресурс захватывается в конструкторе, освобождается в деструкторе

Умные указатели реализуют RAI, автоматически управляя памятью

Примеры RAI в STL: `std::vector`, `std::string`, `std::fstream`

`std::unique_ptr`

- Умный указатель с уникальным владением
- Объект может принадлежать только одному `unique_ptr`
- При уничтожении автоматически освобождает память
- Не поддерживает копирование, но поддерживает перемещение

```
std::unique_ptr<int> ptr1 = std::make_unique<int>(42);  
std::unique_ptr<int> ptr2 = std::move(ptr1);  
//ptr1 -> nullptr, ptr2 -> 42
```

`get()` - возвращает "сырой" указатель (не передаёт владение)

`release()` - освобождает владение, возвращает сырой указатель

`reset(ptr)` - удаляет текущий элемент, присваивает новый

`operator bool` (приведение к типу `bool`) - проверяет, что в указателе что-то есть (был выполнен `make`)

`std::shared_ptr`

- Умный указатель с разделяемым владением
- Несколько объектов могут указывать на один и тот же ресурс
- Владелец не один - счётчик ссылок
- Память освобождается, когда последний `shared_ptr` выходит из области видимости

```
std::shared_ptr<int> ptr1 = std::make_shared<int>(100);  
std::shared_ptr<int> ptr2 = ptr1;  
std::cout << *ptr2; // out: 100
```

std::weak_ptr

Умный указатель, который не владеет объектом (сам по себе не держит объект, но знает жив ли он)

Не меняет счётчик ссылок

Используется для разрыва циклических ссылок

Может быть сконструирован из `shared_ptr`

Позволяет получить `shared_ptr` при необходимости (если объект ещё жив)

В двусвязном списке, реализованном на `shared_ptr`, возникает зацикливание.

Исправление: заменить тип `left` на `weak_ptr`

Проверка живости объекта через weak_ptr

`lock()` возвращает `shared_ptr`, если объект жив

`expired()` проверяет, уничтожен ли объект

```
if (auto locked = node->right.lock()) {  
    // объект ещё жив  
    std::cout << "Still alive\n";  
} else {  
    std::cout << "Object destroyed\n";  
}
```

Анти-зацикливание в дереве с предком

Храни `parent` с типом `weak_ptr`